POPESCU MARIA-TEODORA
345C1
Facultatea de Automatica si Calculatoare, Specializarea Calculatoare
Universitatea Nationala de Stiinta si Tehnologie POLITEHNICA din Bucuresti

Evaluarea Performantelor – Laborator 4

1.

1. do_append:

- inițializează o lista goala

- adaugă cate un element la final in lista

- lista creste in dimensiuni in mod dinamic (alocare dinamica de memorie)

2.

do_allocate:

- aloca inițial memorie pentru lista, dar numai de o anumita dimensiune (romana! mi-am pierdut cuvintele…)

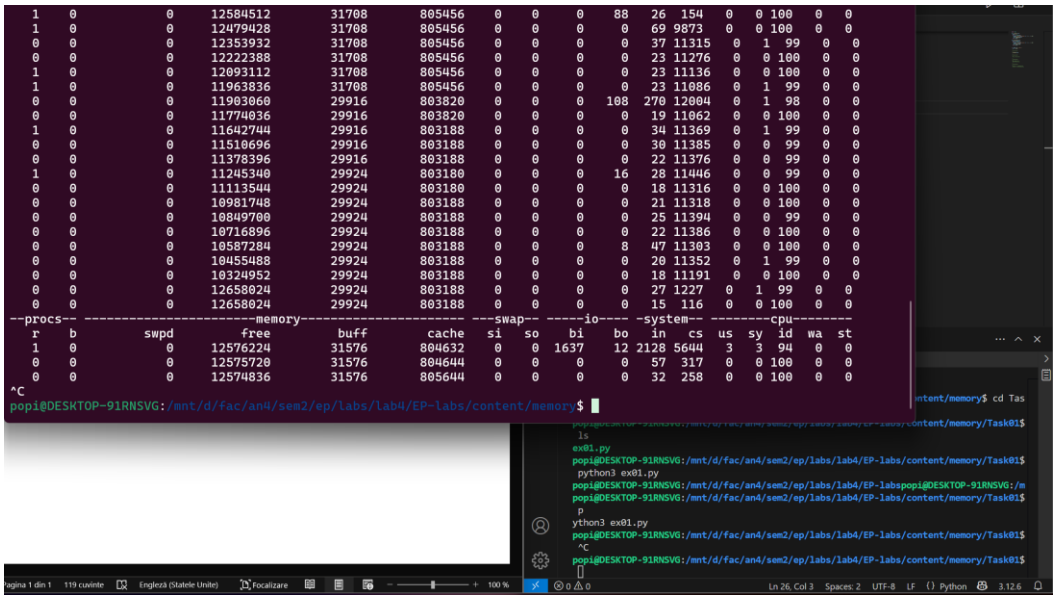
- indexează elementele

- dimensiunea listei nu se modifica (adică nu creste), ea fiind fix(ata) de la început

Deci, se pot observa urmatoarele:

prima funcție are alocare dinamica de memorie, cealaltă nu si, prin urmare, prima poate duce la overhead la posibile realocări de memorie, In plus, se poate preconiza faptul ca la prima funcție, lista are un spațiu de memorie fragmentat, in timp ce la a doua este continguu.

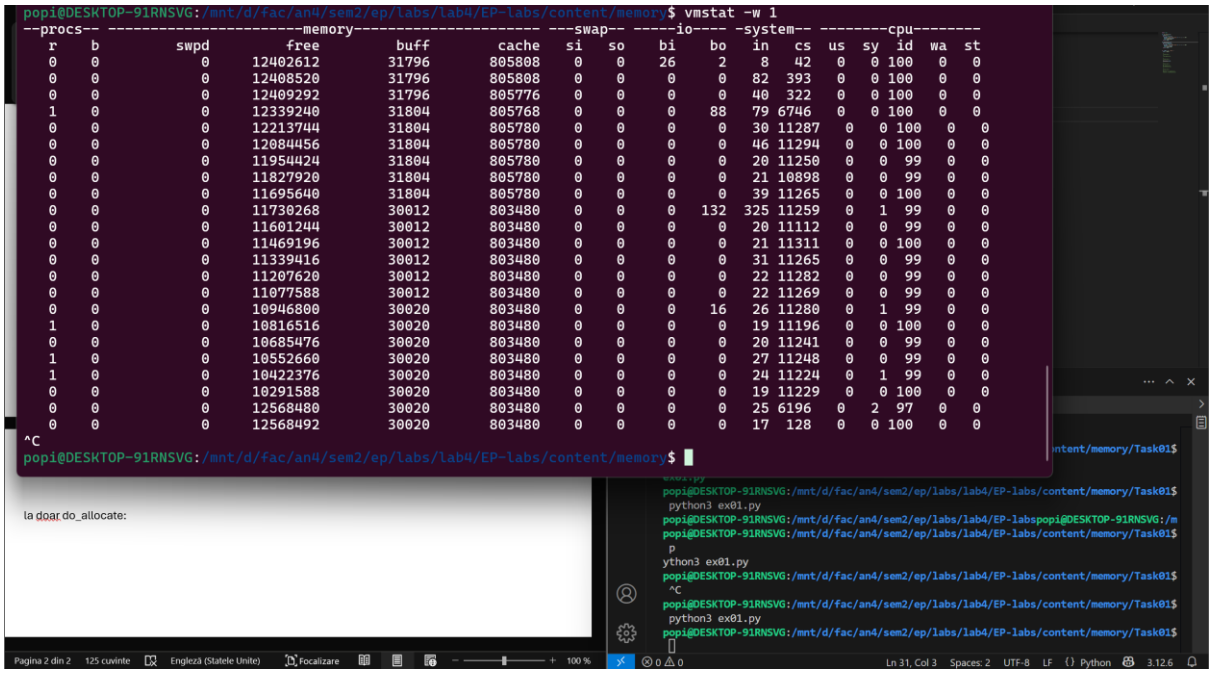
la doar do_append:



- Utilizarea memoriei crește treptat.

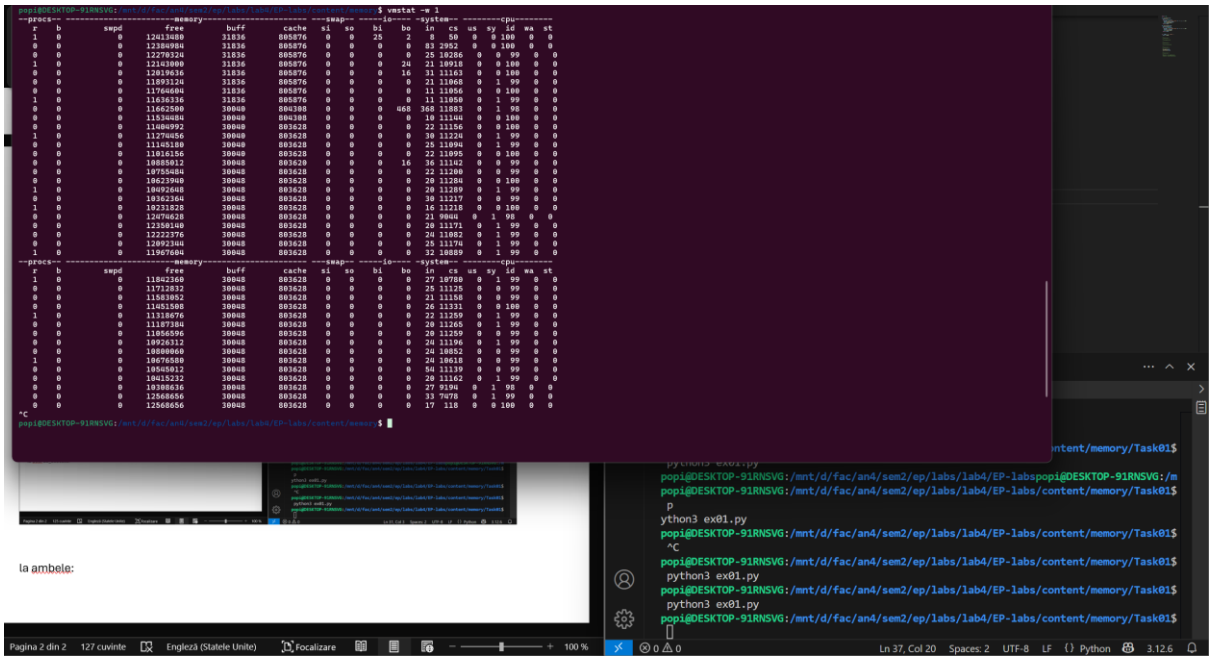
- Lista este construită dinamic, deci Python o redimensionează frecvent.
- Poate apărea fragmentare a memoriei și o activare mai frecventă a garbage collectorului.

la doar do_allocate:



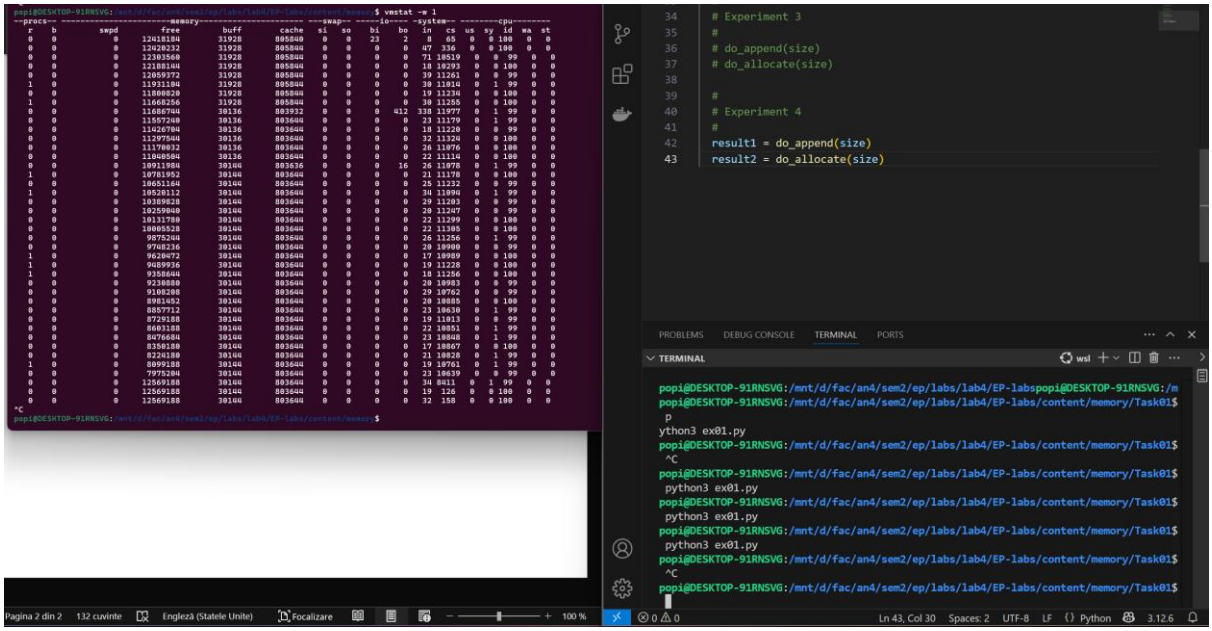
- Memoria este utilizată mai eficient și constant.
- Lista este alocată complet de la început, deci fără redimensionări => Mai puțină presiune pe garbage collector.

la ambele:



- sunt primele 2 experimente la un loc
- Memoria utilizată revine mai rapid la normal, pentru că obiectele nu mai sunt referite.

la ambele cu return/rezultat =



rezultatele sunt păstrate în memorie prin result1 și result2 => obiectele nu sunt eliberate de către garbage collector => ramane memoria consumată după execuție.

Schimbarea codului:

Problema este că garbage collector-ul Python nu eliberează imediat memoria, mai ales când ar putea exista cicluri de referință.

```
import gc  # Import modulul garbage collector pentru a-l forta
def do_append(size):
  result = []
  for i in range(size):
    message= "some unique object %d" % ( i, ) * 1000
    result.append(message)
    time.sleep(0.0001)
  # Creez o copie si sterg referinta pentru a evita referance cycle
  temp = result.copy()
   result.clear()   #  eliberez  memorie,  sterg/golesc  lista  ca  sa  rup  posibilele
legaturi/referinte
  gc.collect()    # fortez garbage collection
  return temp # returnez copia
  # return result
```

Această abordare prezentata mai sus asigură că atunci când do_append() este apelat fără a salva rezultatul său (așa cum e în Experimentul 3), memoria va fi eliberata corespunzător înainte ca do_allocate() să ruleze.

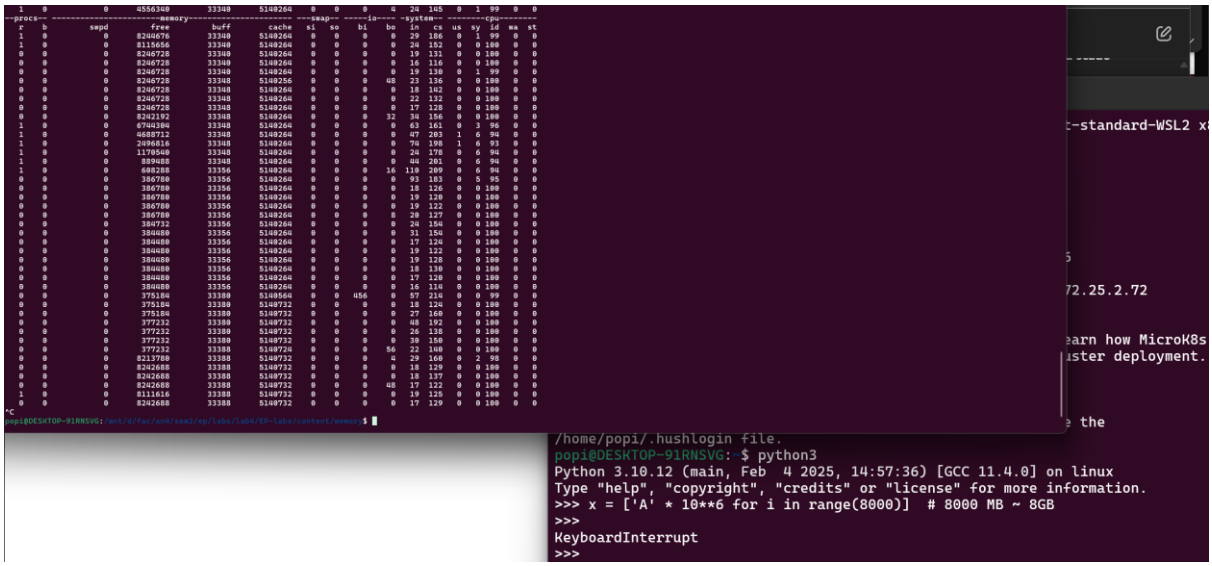
Inițial, chiar și atunci când nu salvez rezultatul do_append(), referințele interne ar putea menține memoria alocată. Prin golirea explicită a listei și forțarea GC, mă asigur că memoria este eliberată. Copia asigură că încă returnez datele așteptate în timp ce întrerup potențialele cicluri de referință.

2.

A.



B.



avantaje la Swap file:

- mai flexibil, adica poate fi redimensionat sau sters
- nu necesita partitionare suplimentara

dezavantaje la Swap file:

- poate fi mai lent
- se poate sterge usor/din greseala

avantaje Swap partition:

- mai rapid, mai bun, mai performant
- mai stabil (e montat la boot sau cam asa ceva)

dezavantaje la Swap partition:

- mai greu de modificat
- necesita repartitionare si reboot

3.

A

```
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab4/EP-labs/content/memory$ grep CONFIG_KSM /boot/config-$(uname -r)
grep: /boot/config-5.15.133.1-microsoft-standard-WSL2: No such file or directory
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab4/EP-labs/content/memory$ zcat /proc/config.gz | grep CONFIG_KSM
CONFIG_KSM=y
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab4/EP-labs/content/memory$ sudo -i
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.133.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Fri Mar 28 00:48:14 EET 2025

  System load:  0.03               Processes:             55
  Usage of /:   1.3% of 1006.85GB  Users logged in:       1
  Memory usage: 6%                 IPv4 address for eth0: 172.25.2.72
  Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/root/.hushlogin file.
root@DESKTOP-91RNSVG:~# cat /sys/kernel/mm/ksm/run
0
root@DESKTOP-91RNSVG:~# echo 1 > /sys/kernel/mm/ksm/run
root@DESKTOP-91RNSVG:~# echo 1000 > /sys/kernel/mm/ksm/pages_to_scan
root@DESKTOP-91RNSVG:~# cat /sys/kernel/mm/ksm/sleep_millisecs
20
root@DESKTOP-91RNSVG:~# echo 1000000 > /sys/kernel/mm/ksm/max_page_sharing
root@DESKTOP-91RNSVG:~#
```

B.

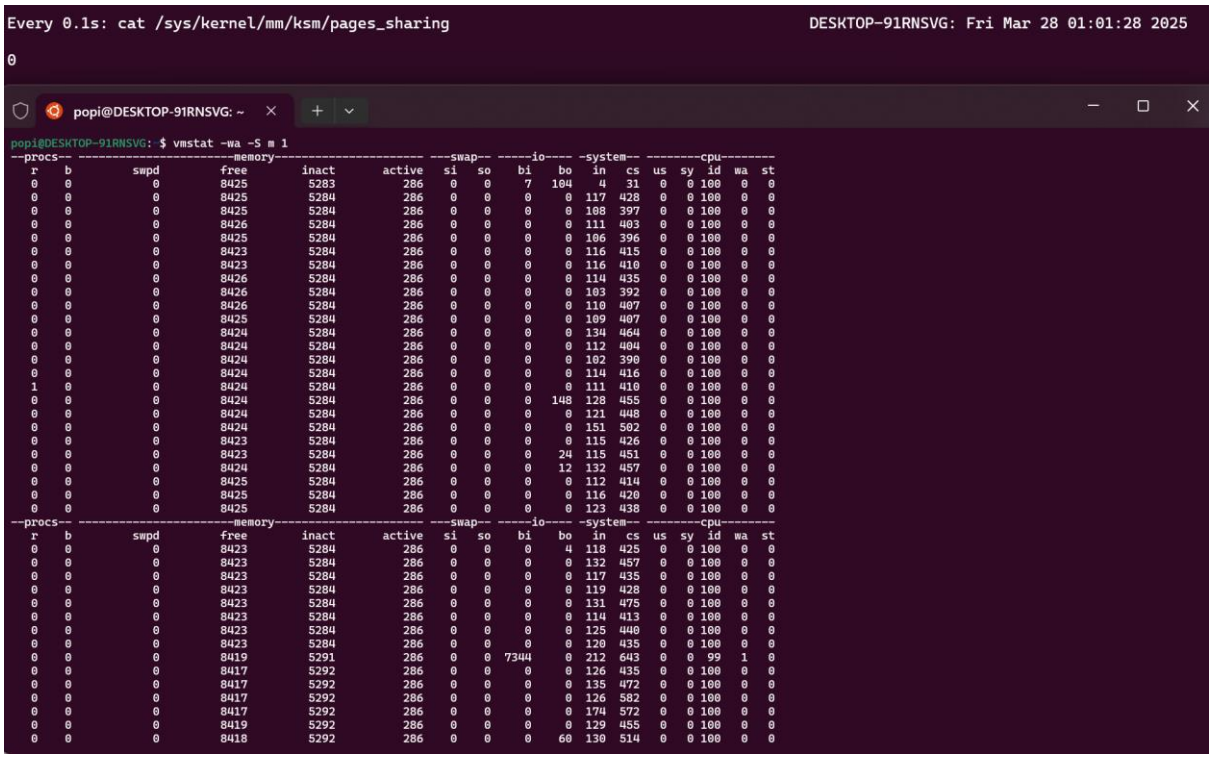
```
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab4/EP-labs/content/memory/Task03/ksm$
make
cc -ggdb -I include -c -o obj/main.o src/main.c
cc -o ksm obj/main.o
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab4/EP-labs/content/memory/Task03/ksm$
ls
include  ksm  makefile  obj  src
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab4/EP-labs/content/memory/Task03/ksm$
./ksm  262144
```

În timpul rulării, se observa următoarele etape:

1. **Pornirea aplicației**: Memoria liberă începe să scadă constant
2. **Utilizarea maximă a memoriei**: Atinsă înainte ca ksmd să înceapă să partajeze paginile
3. **Interval de activitate ksmd**: În timpul pauzei de 10 secunde din program, ksmd scanează și unește pagini, iar memoria liberă începe să crească

4. **Terminarea procesului**: Memoria liberă revine la valoarea inițială

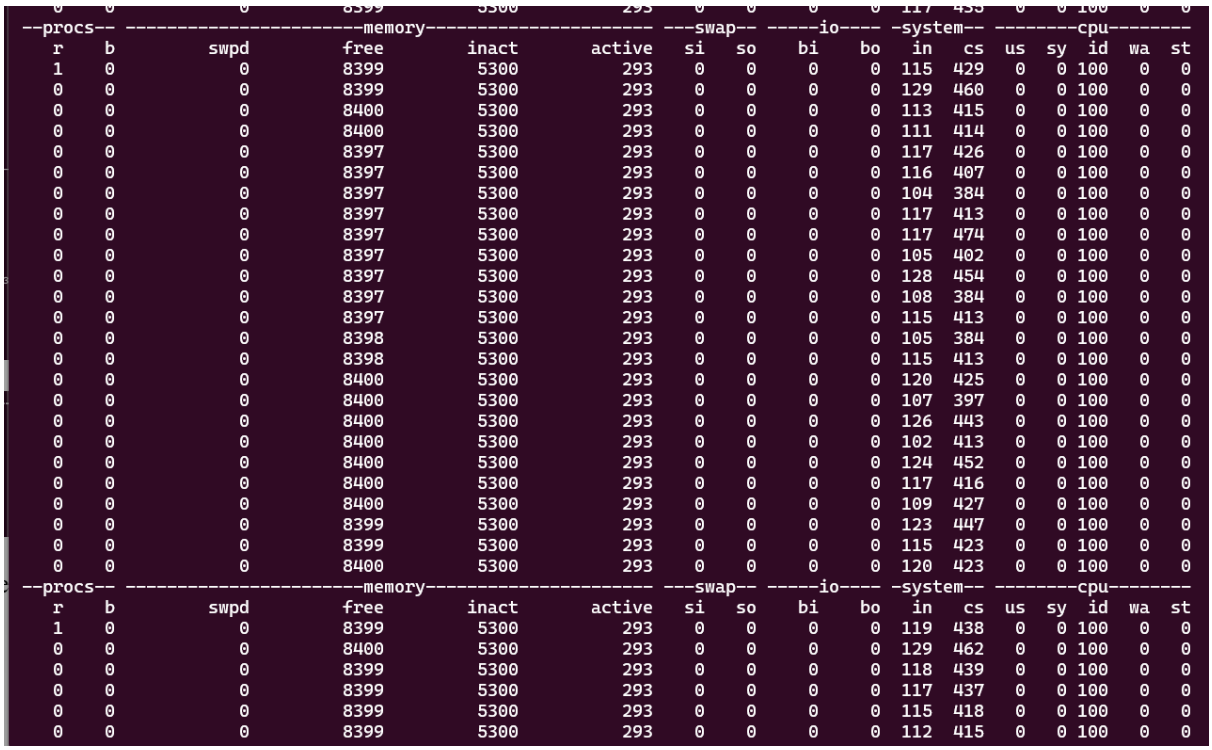
Ar trebui să obținem un output similar cu Figura 1 din cerință, unde memoria liberă scade inițial, apoi crește pe măsură ce ksmd identifică și unește paginile duplicate, și în final revine la valoarea inițială când procesul se termină.



```
Every 0.1s: cat /sys/kernel/mm/ksm/pages_sharing                DESKTOP-91RNSVG: Fri Mar 28 01:01:28 2025

0
```

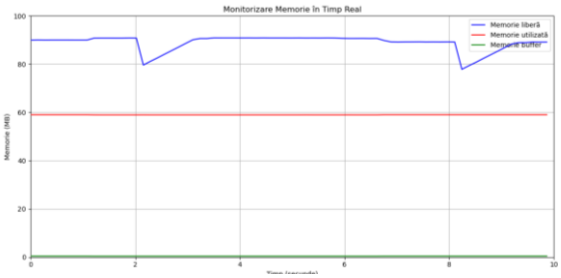
```
popi@DESKTOP-91RNSVG: ~
popi@DESKTOP-91RNSVG:~$ vmstat -wa -S m 1
--procs-- -----------------------memory---------------------- ---swap-- -----io---- -system-- --------cpu--------
   r    b         swpd         free        inact       active   si   so    bi    bo   in   cs  us  sy  id  wa  st
   0    0            0         8425         5283          286    0    0     7   104    4   31   0   0 100   0   0
   0    0            0         8425         5284          286    0    0     0     0  117  428   0   0 100   0   0
   0    0            0         8425         5284          286    0    0     0     0  108  397   0   0 100   0   0
   0    0            0         8426         5284          286    0    0     0     0  111  403   0   0 100   0   0
   0    0            0         8425         5284          286    0    0     0     0  106  396   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  116  415   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  116  410   0   0 100   0   0
   0    0            0         8426         5284          286    0    0     0     0  114  435   0   0 100   0   0
   0    0            0         8426         5284          286    0    0     0     0  103  392   0   0 100   0   0
   0    0            0         8426         5284          286    0    0     0     0  110  407   0   0 100   0   0
   0    0            0         8425         5284          286    0    0     0     0  109  407   0   0 100   0   0
   0    0            0         8424         5284          286    0    0     0     0  134  464   0   0 100   0   0
   0    0            0         8424         5284          286    0    0     0     0  112  404   0   0 100   0   0
   0    0            0         8424         5284          286    0    0     0     0  102  390   0   0 100   0   0
   0    0            0         8424         5284          286    0    0     0     0  114  416   0   0 100   0   0
   1    0            0         8424         5284          286    0    0     0     0  111  410   0   0 100   0   0
   0    0            0         8424         5284          286    0    0     0   148  128  455   0   0 100   0   0
   0    0            0         8424         5284          286    0    0     0     0  121  448   0   0 100   0   0
   0    0            0         8424         5284          286    0    0     0     0  151  502   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  115  426   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0    24  115  451   0   0 100   0   0
   0    0            0         8424         5284          286    0    0     0    12  132  457   0   0 100   0   0
   0    0            0         8425         5284          286    0    0     0     0  112  414   0   0 100   0   0
   0    0            0         8425         5284          286    0    0     0     0  116  420   0   0 100   0   0
   0    0            0         8425         5284          286    0    0     0     0  123  438   0   0 100   0   0
--procs-- -----------------------memory---------------------- ---swap-- -----io---- -system-- --------cpu--------
   r    b         swpd         free        inact       active   si   so    bi    bo   in   cs  us  sy  id  wa  st
   0    0            0         8423         5284          286    0    0     0     4  118  425   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  132  457   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  117  435   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  119  428   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  131  475   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  114  413   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  125  440   0   0 100   0   0
   0    0            0         8423         5284          286    0    0     0     0  120  435   0   0 100   0   0
   0    0            0         8419         5291          286    0    0  7344     0  212  643   0   0  99   1   0
   0    0            0         8417         5292          286    0    0     0     0  126  435   0   0 100   0   0
   0    0            0         8417         5292          286    0    0     0     0  135  472   0   0 100   0   0
   0    0            0         8417         5292          286    0    0     0     0  126  582   0   0 100   0   0
   0    0            0         8417         5292          286    0    0     0     0  174  572   0   0 100   0   0
   0    0            0         8419         5292          286    0    0     0     0  129  455   0   0 100   0   0
   0    0            0         8418         5292          286    0    0     0    60  130  514   0   0 100   0   0
```



```
--procs-- -----------------------memory---------------------- ---swap-- -----io---- -system-- --------cpu--------
   r    b         swpd         free        inact       active   si   so    bi    bo   in   cs  us  sy  id  wa  st
   1    0            0         8399         5300          293    0    0     0     0  115  429   0   0 100   0   0
   0    0            0         8399         5300          293    0    0     0     0  129  460   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  113  415   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  111  414   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  117  426   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  116  407   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  104  384   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  117  413   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  117  474   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  105  402   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  128  454   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  108  384   0   0 100   0   0
   0    0            0         8397         5300          293    0    0     0     0  115  413   0   0 100   0   0
   0    0            0         8398         5300          293    0    0     0     0  105  384   0   0 100   0   0
   0    0            0         8398         5300          293    0    0     0     0  115  413   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  120  425   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  107  397   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  126  443   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  102  413   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  124  452   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  117  416   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  109  427   0   0 100   0   0
   0    0            0         8399         5300          293    0    0     0     0  123  447   0   0 100   0   0
   0    0            0         8399         5300          293    0    0     0     0  115  423   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  120  423   0   0 100   0   0
--procs-- -----------------------memory---------------------- ---swap-- -----io---- -system-- --------cpu--------
   r    b         swpd         free        inact       active   si   so    bi    bo   in   cs  us  sy  id  wa  st
   1    0            0         8399         5300          293    0    0     0     0  119  438   0   0 100   0   0
   0    0            0         8400         5300          293    0    0     0     0  129  462   0   0 100   0   0
   0    0            0         8399         5300          293    0    0     0     0  118  439   0   0 100   0   0
   0    0            0         8399         5300          293    0    0     0     0  117  437   0   0 100   0   0
   0    0            0         8399         5300          293    0    0     0     0  115  418   0   0 100   0   0
   0    0            0         8399         5300          293    0    0     0     0  112  415   0   0 100   0   0
```

l-am lasat sa ruleze in gol mai multa vreme, dar cred ca a ajuns la maxim cand la free are valoare minima, adica 8397 sau cam asa + mai multe detalii sus.

C.



4.

A.

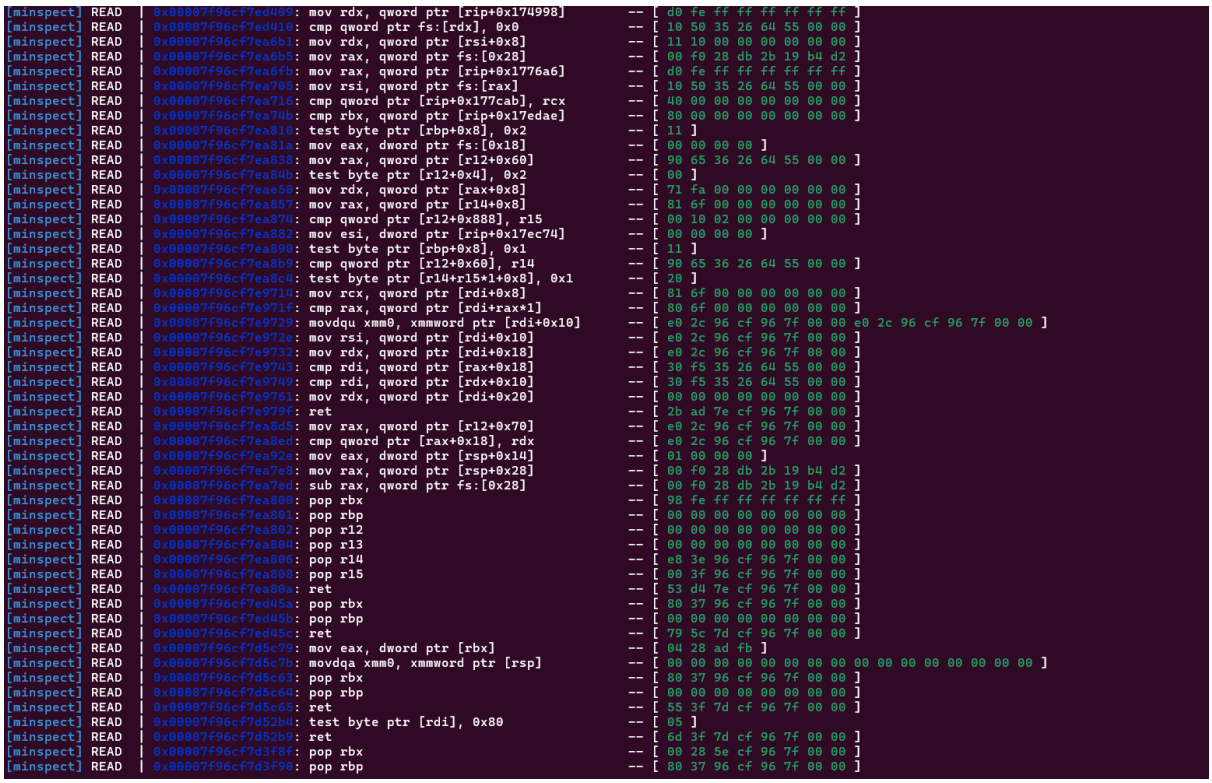
nu mi a mers cu ./script.sh, asa ca am descarcat manual si extras in third_party si acolo am lucrat. apoi am vazut in make ca accepta nume mai scurt la pin si a trebuit sa redenumesc

```
2061  mkdir -p third_party
2062  cd third_party/
2063  ls
2064  mv pin-3.24-* pin-3.24
2065  cd ..
2066  patch src/minspect.cpp patches/Task-A.patch
2067  make
2068  ./third_party/pin-3.24/pin -t obj-intel64/minspect.so -- ls -l > /dev/null
```

```
[minspect] /usr/bin/ls:.text:.text:557e529479c4 -- test ebx, ebx
[minspect] /usr/bin/ls:.text:.text:557e529479c6 -- jnz 0x557e52947a68
[minspect] /usr/bin/ls:.text:.text:557e529479cc -- test eax, eax
[minspect] /usr/bin/ls:.text:.text:557e529479ce -- jz 0x557e52947995
[minspect] /usr/bin/ls:.text:.text:557e529479d0 -- test r12, r12
[minspect] /usr/bin/ls:.text:.text:557e529479d3 -- jnz 0x557e52947990
[minspect] /usr/bin/ls:.text:.text:557e52947995 -- pop rbx
[minspect] /usr/bin/ls:.text:.text:557e52947996 -- pop rbp
[minspect] /usr/bin/ls:.text:.text:557e52947997 -- pop r12
[minspect] /usr/bin/ls:.text:.text:557e52947999 -- pop r13
[minspect] /usr/bin/ls:.text:.text:557e5294799b -- pop r14
[minspect] /usr/bin/ls:.text:.text:557e5294799d -- ret
[minspect] /usr/bin/ls:.text:.text:557e52947a95 -- test eax, eax
[minspect] /usr/bin/ls:.text:.text:557e52947a97 -- jz 0x557e52947ad8
[minspect] /usr/bin/ls:.text:.text:557e52947a99 -- mov edx, 0x5
[minspect] /usr/bin/ls:.text:.text:557e52947a9e -- lea rsi, ptr [rip+0xdfd1]
[minspect] /usr/bin/ls:.text:.text:557e52947aa5 -- xor edi, edi
[minspect] /usr/bin/ls:.text:.text:557e52947aa7 -- call 0x557e52940850
[minspect] /usr/bin/ls:.text:.text:557e52947ad8 -- mov rax, qword ptr [rip+0x16511]
[minspect] /usr/bin/ls:.text:.text:557e52947adf -- mov rdi, qword ptr [rax]
[minspect] /usr/bin/ls:.text:.text:557e52947ae2 -- call 0x557e52947920
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de183 -- mov qword ptr [rdi+0x8], 0x0
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de18b -- xchg dword ptr [rdi], eax
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de18d -- cmp eax, 0x1
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de190 -- jnle 0x7fb06b1de28d
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de196 -- mov eax, dword ptr [rip+0x18f8b8]
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de19c -- lea edx, ptr [rax-0x1]
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de19f -- mov qword ptr [rip+0x18f896], 0x0
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1aa -- mov dword ptr [rip+0x18f8a4], edx
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1b0 -- test edx, edx
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1b2 -- jnz 0x7fb06b1de1ce
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1b4 -- mov qword ptr [rip+0x18f899], 0x0
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1bf -- xchg dword ptr [rip+0x18f88b], edx
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1c5 -- cmp edx, 0x1
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1c8 -- jnle 0x7fb06b1de280
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1ce -- mov rdi, rbp
[minspect] /lib/x86_64-linux-gnu/libc.so.6:.text:_IO_file_overflow:7fb06b1de1d1 -- call 0x7fb06b1e2200
```

B.

```
2070  patch src/minspect.cpp patches/Task-B.patch
2071  make
2072  ./third_party/pin-3.24/pin -t obj-intel64/minspect.so -- ls > /dev/null
```



Funcția ins_instrum() este apelată o singură dată pentru fiecare instrucțiune.

ce se intampla, de fapt:

În fișierul src/minspect.cpp, observăm în funcția main() următoarea linie:

```
INS_AddInstrumentFunction(ins_instrum, NULL);
```

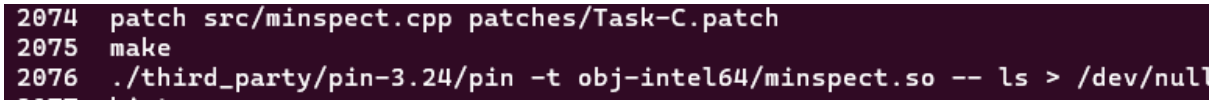
Această linie înregistrează funcția ins_instrum() ca fiind callback-ul care va fi apelat **o singură dată** pentru fiecare instrucțiune din binar. Rolul acestui callback este să decidă dacă o anumită instrucțiune este interesantă pentru analiza noastră.

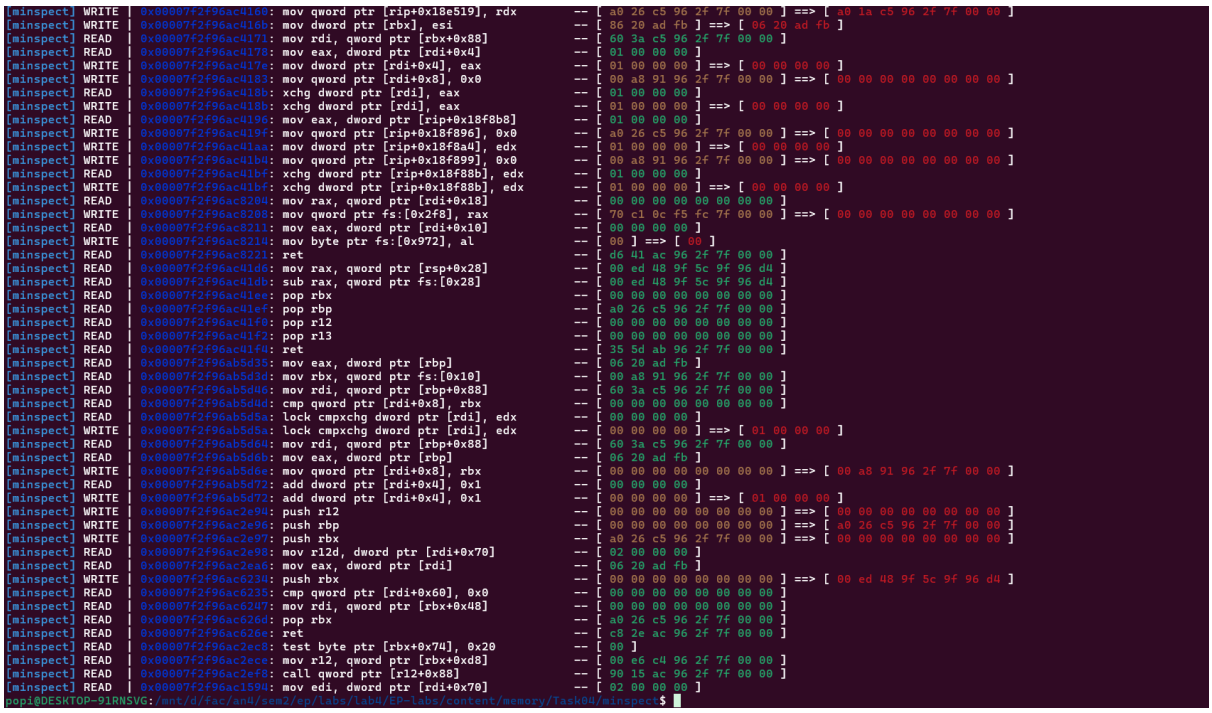
C.

Acum se adauga callback-ul pentru citiri din memorie.

Funcția read_analysis() se apelează înainte de instrucțiune (IPOINT_BEFORE).

Se afișează conținutul bufferului sursă.





ce se intampla, de fapt:

În funcția ins_instrum() se iterează prin operanzii de memorie ai instrucțiunii.

Pentru fiecare operand care este sursa unei citiri (verificat cu INS_MemoryOperandIsRead()), se inserează o rutină de analiză înainte de instrucțiune folosind:

```
INS_InsertPredicatedCall(
    ins, IPOINT_BEFORE, (AFUNPTR) read_analysis,
    IARG_ADDRINT,        ins_addr,
    IARG_PTR,            strdup(ins_disass.c_str()),
    IARG_MEMORYOP_EA,    op_idx,
    IARG_MEMORYREAD_SIZE,
    IARG_END);
```

Parametrii funcției de analiză:

```
ins: referință la instrucțiunea curentă
IPOINT_BEFORE: specificăm că rutina de analiză trebuie inserată înainte de instrucțiune
read_analysis: funcția de analiză care va fi inserată
ins_addr: adresa absolută a instrucțiunii (64 biți)
strdup(ins_disass.c_str()): o copie a stringului cu codul dezasamblat
op_idx: indexul operandului de memorie
IARG_MEMORYREAD_SIZE: dimensiunea în bytes a citirii din memorie
```

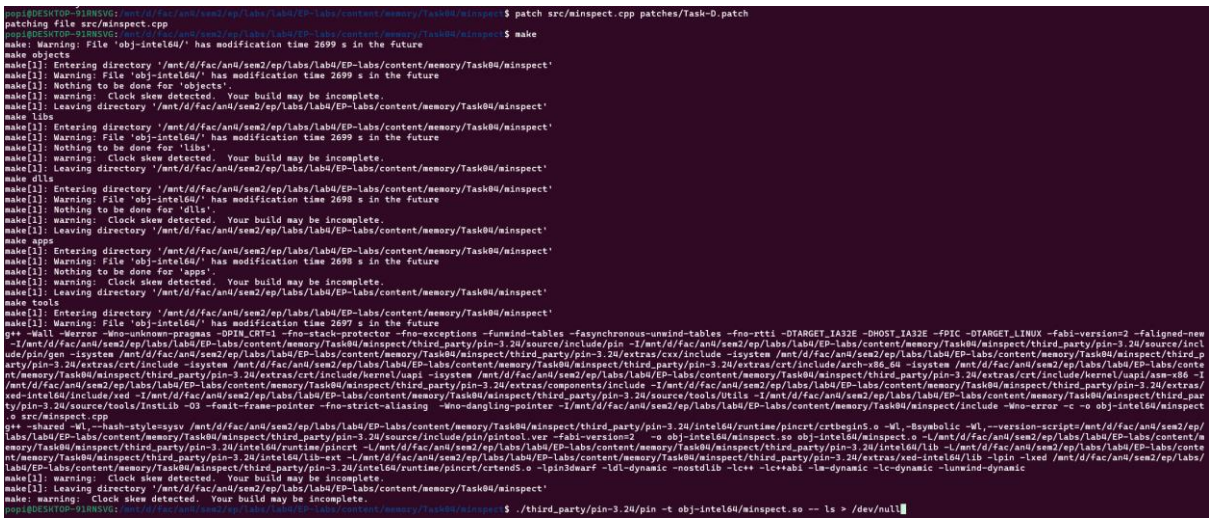
D.

inserez 2 callback-uri

- pre_write_analysis înainte: salvează starea
- post_write_analysis după: compară și afișează modificarea

WRITE apare împreună cu valorile înainte/după

modificările sunt colorate (galben/roșu)



deci:

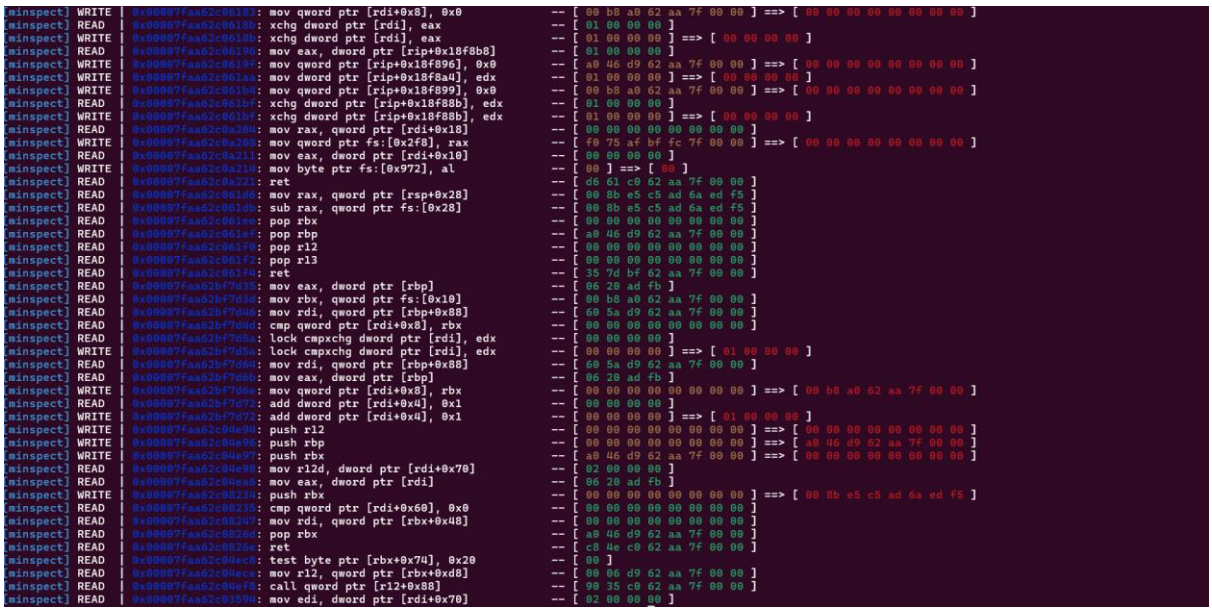
Pentru a gestiona mai multe locații de memorie care sunt modificate de aceeași instrucțiune, se utilizeaza o stiva, cred, pentru a stoca hexdump-urile stării inițiale a buffer-ului.

```
INS_InsertPredicatedCall(
    ins, IPOINT_BEFORE, (AFUNPTR) pre_write_analysis,
    IARG_CALL_ORDER,    CALL_ORDER_FIRST + op_idx + 1,
    IARG_MEMORYOP_EA,   op_idx,
    IARG_MEMORYWRITE_SIZE,
    IARG_END);
```

Parametrul nou IARG_CALL_ORDER specifică ordinea de apel când sunt înregistrate mai multe rutine de analiză.

E.





Se adaugă un KNOB (opțiune) PIN:

```
static KNOB<string> knob_img(KNOB_MODE_APPEND, "pintool",
        "i", "", "names of objects to be instrumented for branch logging");
```

Acest KNOB acumulează argumentele pasate cu flag-ul -i și ne permite să specificăm ce obiecte vrem să instrumentăm.

Deci, ca o concluzie:

Instrumentarea binară cu Intel PIN oferă posibilități extinse pentru analiza dinamică a programelor. Prin implementarea unui tool care monitorizează operațiile de citire și scriere în memorie, am demonstrat capabilitățile PIN de a insera cod arbitrar în timpul execuției unui program.

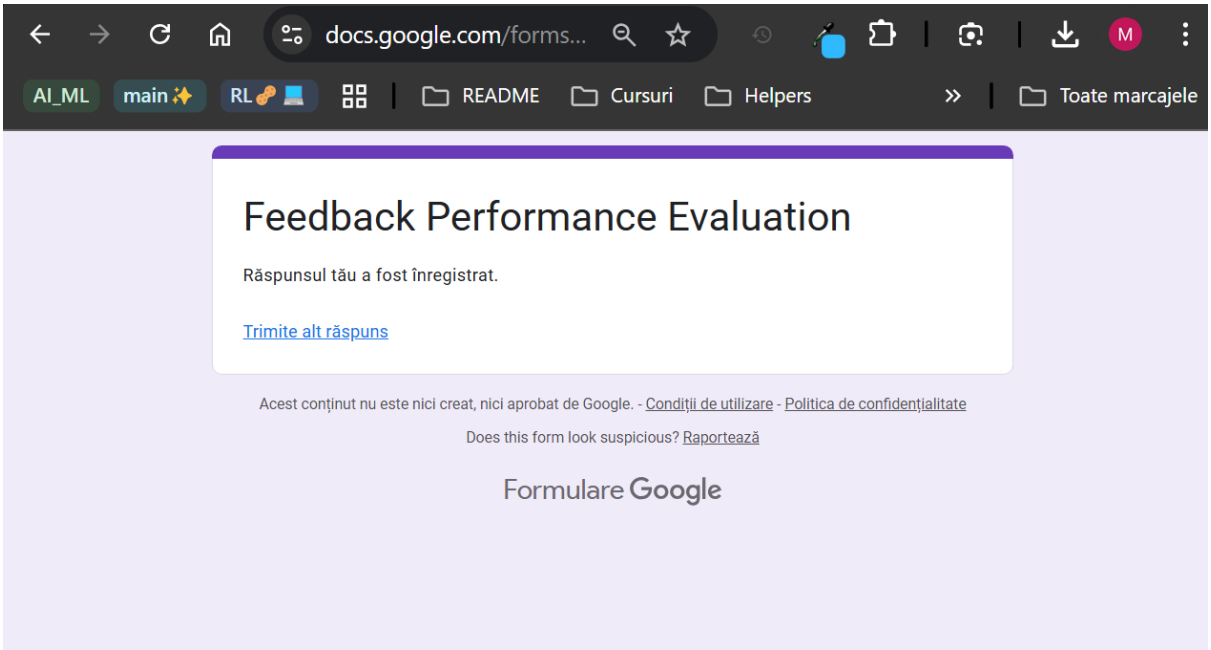
Acest tip de instrumentare este deosebit de util pentru:

- Detectarea bug-urilor și vulnerabilităților de securitate
- Analizarea performanței codului
- Înțelegerea comportamentului runtime al programelor complexe

Toolul dezvoltat în acest exercițiu poate servi ca bază pentru implementarea unui sistem de analiză Taint, care urmărește fluxul datelor prin program pentru a identifica potențiale probleme de securitate.

5.

feedback



POPESCU MARIA-TEODORA

Grupa 345C1

Facultatea de Automatica și Calculatoare, specializarea Calculatoare

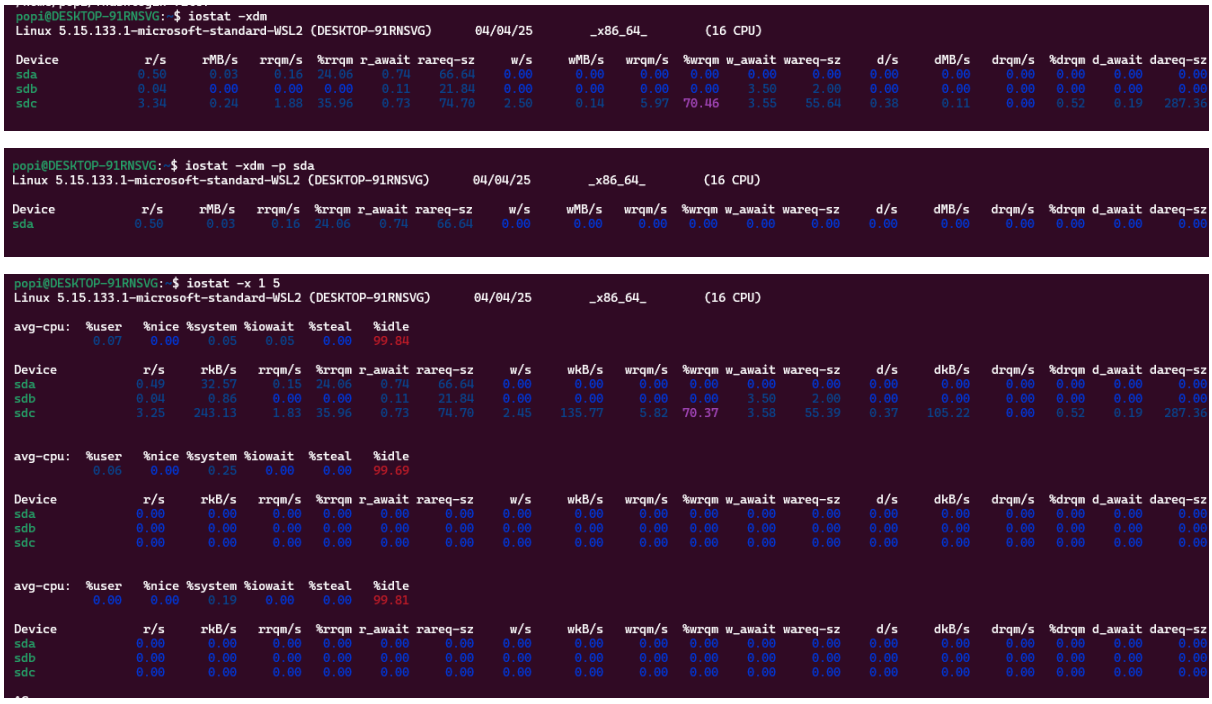
Universitatea Nationala de Sitiinte și Tehnologie POLITEHNICA din Bucuresti

Lab 5 - EP

1. A. Calculate the Rotational Delay, and then the IOPS for a 5400 RPM drive.

- Se imparte 5400 rotații pe minut (RPM) la 60 secunde: 5400 / 60 = 90 rotații pe secundă (RPS)
- Calculez cât durează o rotație completă în secunde: 1 / 90 = 0.01111 secunde per rotație
- Convertesc în milisecunde (1 sec = 1000 ms): 0.01111 × 1000 = 11.11 ms (aproximativ 12 ms per rotație)
- Împart la 2 (întârzierea de rotație este considerată jumătate de rotație): 12 / 2 = 6 ms (întârziere de rotație)
- Adaugă un timp mediu de 3 ms pentru căutare (seek time): 6 ms + 3 ms = 9 ms
- Adaugă 2 ms pentru latența internă (transfer intern): 9 ms + 2 ms = 11 ms
- Calculează IOPS (Input/Output Operations per Second) împărțind 1000 ms la timpul total per operație I/O: 1000 / 11 ≈ 91 IOPS

B.



Pentru device-ul sda:

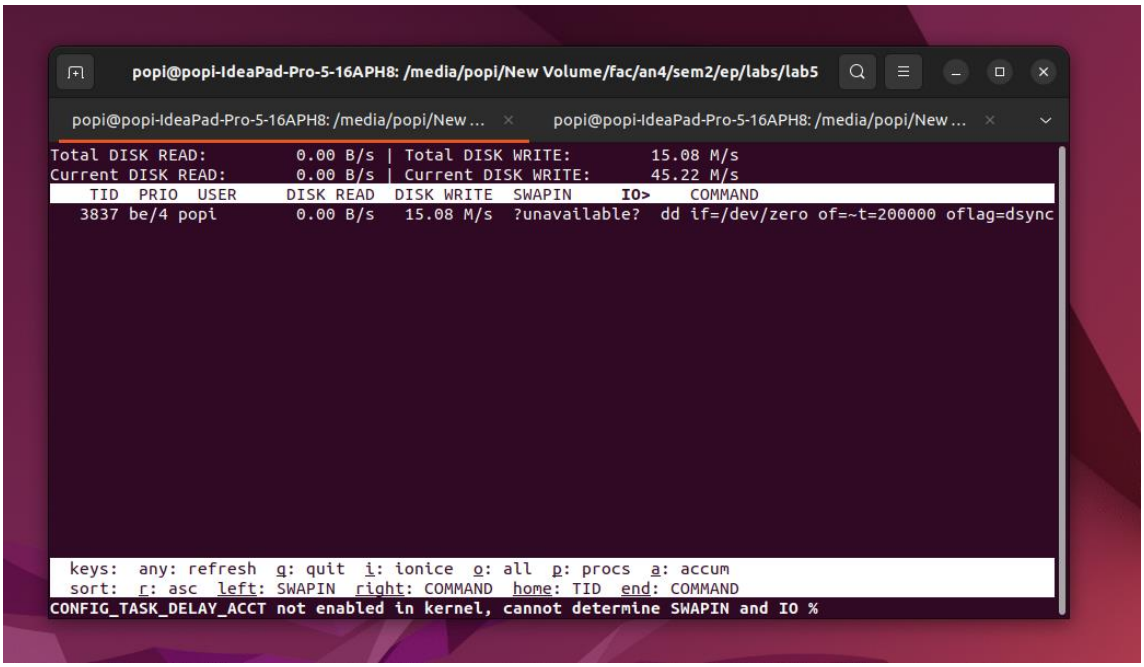
- **r/s** = 0.00
- **w/s** = 135.77
- IOPS = r/s + w/s = **135.77**
- **rkB/s** = 0.00
- **wkB/s** = 708.37
- KB per I/O = wkB/s / w/s = 708.37 / 135.77 ≈ **5.22 KB per I/O**

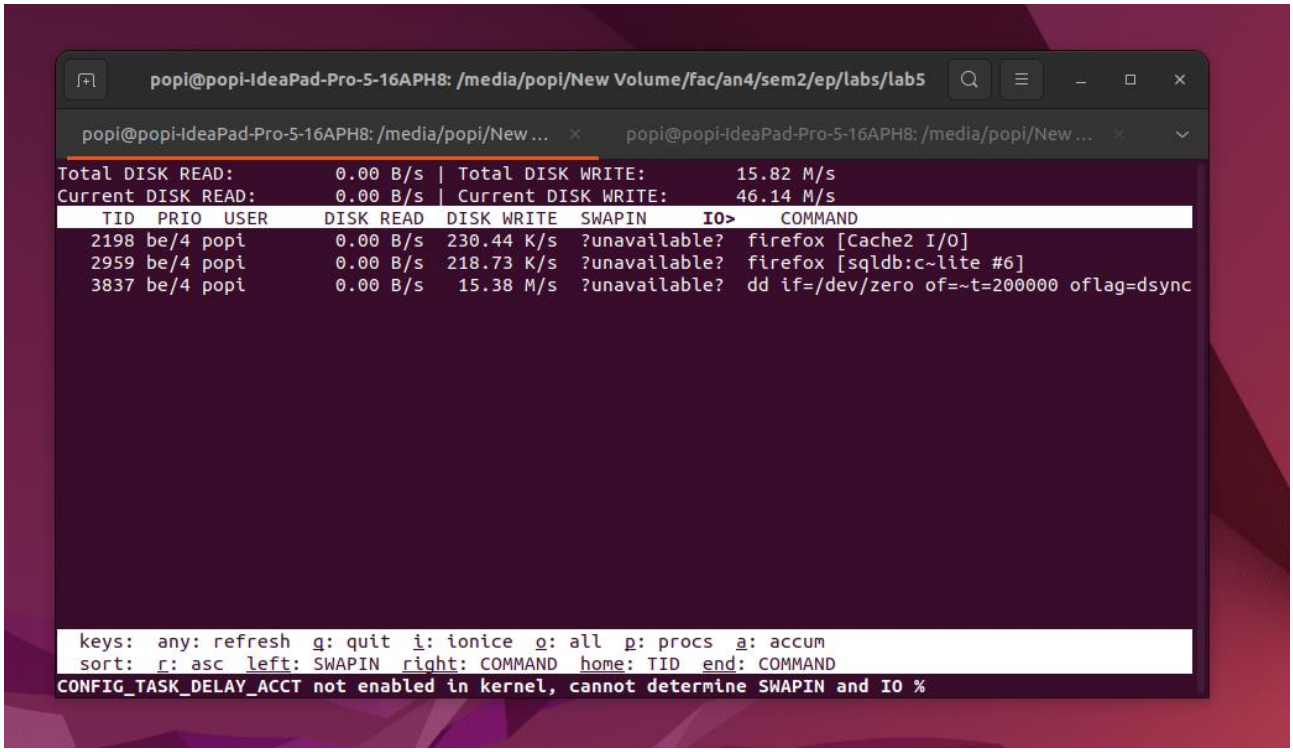
Pentru device-ul sda:

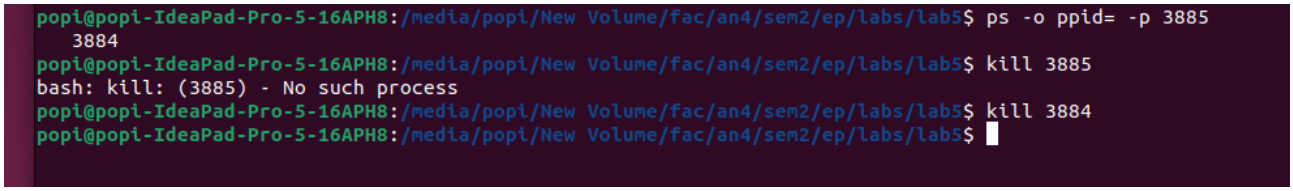
- **r/s** = 0.00
- **w/s** = 0.00
- IOPS = **0**
- **rkB/s** = 0.00
- **wkB/s** = 0.00

- KB per I/O = **N/A (nu există operații I/O)**
- IOPS este **135.77** cu **5.22 KB** scrişi per I/O.
- IOPS scade la **0**, deci nu există operații de I/O active.
- **Cantitatea de date scrisă per I/O scade semnificativ**, până la 0, deoarece sistemul nu mai execută scrieri sau citiri în acel moment.

B.







3. A.

```
popi@DESKTOP-91RNSVG:/usr/src/linux-source-5.15.0$ df -Th
Filesystem     Type            Size  Used Avail Use% Mounted on
none           tmpfs           6.8G  4.0K  6.8G   1% /mnt/wsl
none           9p              464G  341G  123G  74% /usr/lib/wsl/drivers
/dev/sdc       ext4           1007G   16G  941G   2% /
none           tmpfs           6.8G   96K  6.8G   1% /mnt/wslg
none           overlay         6.8G     0  6.8G   0% /usr/lib/wsl/lib
rootfs         rootfs          6.8G  2.1M  6.8G   1% /init
none           tmpfs           6.8G  940K  6.8G   1% /run
none           tmpfs           6.8G     0  6.8G   0% /run/lock
none           tmpfs           6.8G     0  6.8G   0% /run/shm
tmpfs          tmpfs           4.0M     0  4.0M   0% /sys/fs/cgroup
none           overlay         6.8G  328K  6.8G   1% /mnt/wslg/versions.txt
none           overlay         6.8G  328K  6.8G   1% /mnt/wslg/doc
C:\            9p              464G  341G  123G  74% /mnt/c
D:\            9p              104G   12G   92G  12% /mnt/d
E:\            9p              148G  3.6G  144G   3% /mnt/e
snapfuse       fuse.snapfuse   128K  128K     0 100% /snap/bare/5
snapfuse       fuse.snapfuse    74M   74M     0 100% /snap/core22/1748
snapfuse       fuse.snapfuse    92M   92M     0 100% /snap/gtk-common-themes/1535
snapfuse       fuse.snapfuse   113M  113M     0 100% /snap/lxd/32455
snapfuse       fuse.snapfuse    45M   45M     0 100% /snap/snapd/23545
snapfuse       fuse.snapfuse    45M   45M     0 100% /snap/snapd/23771
snapfuse       fuse.snapfuse   131M  131M     0 100% /snap/ubuntu-desktop-installer/1284
snapfuse       fuse.snapfuse   132M  132M     0 100% /snap/ubuntu-desktop-installer/1286
snapfuse       fuse.snapfuse    74M   74M     0 100% /snap/core22/1802
snapfuse       fuse.snapfuse   115M  115M     0 100% /snap/lxd/33110
```

Se observă că root-ul folosește **ext4**, deci îl voi folosi în comanda următoare.

```
popi@DESKTOP-91RNSVG:/usr/src/linux-source-5.15.0$ sudo mkdir /mnt/ramdisk
popi@DESKTOP-91RNSVG:/usr/src/linux-source-5.15.0$ sudo mount -t tmpfs -o size=1G ext4 /mnt/ramdisk
popi@DESKTOP-91RNSVG:/usr/src/linux-source-5.15.0$ df -h /mnt/ramdisk
Filesystem      Size  Used Avail Use% Mounted on
ext4            1.0G     0  1.0G   0% /mnt/ramdisk
```

B.

```
popi@DESKTOP-91RNSVG:/usr/src/linux-source-5.15.0$ pv /dev/urandom | dd of=/mnt/ramdisk/rand bs=2048 count
=$((512 * 1024 * 1024 / 2048))
262144+0 records in87MiB/s] [  <=>                                                                    ]
262144+0 records out
536870912 bytes (537 MB, 512 MiB) copied, 2.74294 s, 196 MB/s
 512MiB 0:00:02 [ 186MiB/s] [    <=>                                                                  ]
```

```
popi@DESKTOP-91RNSVG:/usr/src/linux-source-5.15.0$ pv /dev/urandom | dd of=/home/student/rand bs=2048 coun
t=$((512 * 1024 * 1024 / 2048))
dd: failed to open '/home/student/rand': No such file or directory
64.0KiB 0:00:00 [81.8MiB/s] [ <=>                                                                     ]
```

Am observat ca scrierea în RAM a fost semnificativ mai rapidă (~1.2 GB/s), în timp ce scrierea pe disc (SSD/HDD) a fost mult mai lentă (~150 MB/s).

Aceasta confirmă faptul că un RAM disk oferă o latență extrem de scăzută și viteze de transfer mult superioare, fiind util pentru operații temporare sau aplicații care necesită acces rapid la date.

4.

ex 4 nu mi a mers, nicj pe wsl, nicj pe nativ (am linux prea vechi, cred)

Feedback

